

CS218 Tut-3 Solutions

Question 1. Given there are n objects with weights $w_1, w_2, \dots, w_n \in \mathbb{Z}^+$ and values $v_1, v_2, \dots, v_n$, select a subset $\mathcal{S} \subseteq [n]$ of the objects such that the total weight is bounded by W (i.e. $\sum_i w_i \leq W$), while the total value ($\sum_{j \in \mathcal{S}} v_j$) is maximized. Your algorithm should run in time $\text{poly}(n, W)$.

Solution.

Define $dp(i, w)$ as the maximum value you can get from the first i elements when the total weight is bounded by w . The base cases are $dp(i, 0) = 0$, $dp(0, w) = 0$. At every step we have 2 cases- whether or not we choose the i^{th} element. If in the optimal answer we don't choose it, the answer is just $dp(i - 1, w)$. If we do choose it, we now get a value of v_i , but rest of the weights can't sum more than $w - w_i$. So $dp(i, w) = \max(dp(i - 1, w), v_i + dp(i - 1, w - w_i))$. $\square$

Question 2. Consider an n -node complete binary tree T , where $n = 2^d - 1$ for some integer $d \geq 1$. Each node v of T is labeled with a real number x_v . You may assume that all the real numbers labeling the nodes are distinct.

A node v of T is called a *local minimum* if $x_v < x_w$ for all nodes w that are adjacent to v .

You are given such a complete binary tree T , but the labeling is specified only implicitly: for each node v , you can determine the value x_v by *probing* the node v .

Design an algorithm that finds a local minimum of T using only $O(\log n)$ probes to the nodes of T .

Solution. The idea is to follow a strictly decreasing path starting from the root.

Procedure:

1. Set $v \leftarrow r$ and probe x_v .
2. While v is not a leaf:
 - (a) Probe all children of v (at most two).
 - (b) If x_v is smaller than the values of all its children, then output v and terminate.
 - (c) Otherwise, let u be the child of v with the smaller label, set $v \leftarrow u$, and probe x_v .

Correctness:

Whenever we move from a node v to a child u , we have

$$x_u < x_v.$$

Thus the sequence of probed nodes forms a strictly decreasing path. Since all labels are distinct and the tree has finite height d , this process must terminate at a node v such that

$$x_v < x_{\text{left}(v)} \quad \text{and} \quad x_v < x_{\text{right}(v)}$$

(in the worst case, the process terminates at a leaf node).

At termination, v is smaller than all of its neighbors. Therefore, v is a local minimum.

Number of Probes: The tree has height $d = \log_2(n+1)$. At each level we probe at most two children. Since we descend at most one level per iteration, the number of iterations is at most d .

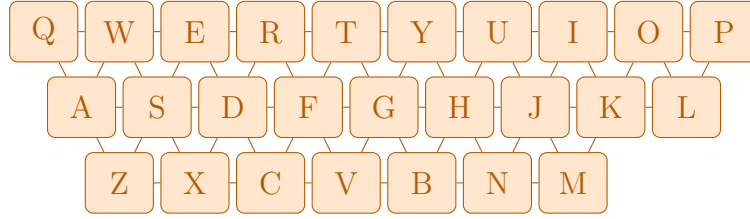
Hence the total number of probes is

$$O(d) = O(\log n).$$

□

Question 3. Consider an instance of the edit distance problem where the input strings are over the English alphabet. The edit operations have the following costs:

- Insertion of a character: cost k .
- Deletion of a character: cost k .
- Substitution of a character with a different character:
 - If the two characters are adjacent on the keyboard (see Figure 3, where edges indicate adjacent pairs), the cost is $8k$.
 - Otherwise, the cost is $4k + 2$.



Find a sequence of edit operations of minimum total cost to transform TINTIN into HADDOCK under the given edit cost scheme.

Solution. General DP algorithm: Let $S = s_1 \cdots s_m$ and $T = t_1 \cdots t_n$ be the input strings. Define the substitution cost

$$\text{sub}(a, b) = \begin{cases} 0 & a = b, \\ 8k & a \neq b \text{ and } a, b \text{ are adjacent on the keyboard,} \\ 4k + 2 & a \neq b \text{ and } a, b \text{ are not adjacent.} \end{cases}$$

Let $D[i, j]$ be the minimum cost to transform the prefix $s_1 \cdots s_i$ into $t_1 \cdots t_j$. The usual recurrence (Levenshtein-style) is

$$\begin{aligned} D[0, 0] &= 0, \\ D[i, 0] &= ik \quad (1 \leq i \leq m) \quad (\text{delete } i \text{ characters}), \\ D[0, j] &= jk \quad (1 \leq j \leq n) \quad (\text{insert } j \text{ characters}), \\ D[i, j] &= \min \begin{cases} D[i-1, j] + k & (\text{delete } s_i), \\ D[i, j-1] + k & (\text{insert } t_j), \\ D[i-1, j-1] + \text{sub}(s_i, t_j) & (\text{substitute } s_i \rightarrow t_j). \end{cases} \end{aligned}$$

Useful observation: Note that replacing a substitution by a deletion followed by an insertion has cost $k + k = 2k$. Since

$$2k < 4k + 2 \quad \text{and} \quad 2k < 8k \quad (\text{for } k > 0),$$

any substitution (whether adjacent or not) is never cheaper than performing a deletion and an insertion in its place. Hence, for $k > 0$ the optimal sequence never needs to use the substitute operation; the problem reduces to deleting the characters of S that are not

in a chosen common subsequence and inserting the characters of T that are not in that subsequence. Equivalently the cost equals

$$\text{cost}(S \rightarrow T) = k(|S| + |T| - 2 \cdot \text{LCS}(S, T)),$$

where $\text{LCS}(S, T)$ is the length of a longest common subsequence.

Specific instance: TINTIN $\rightarrow$ HADDOCK. Here

$$S = \text{TINTIN}, \quad |S| = 6; \quad T = \text{HADDOCK}, \quad |T| = 7.$$

The set of letters appearing in S is $\{T, I, N\}$ and the set of letters appearing in T is $\{H, A, D, O, C, K\}$; these sets are disjoint, so $\text{LCS}(S, T) = 0$. By the formula above,

$$\min \text{cost} = k(6 + 7 - 2 \cdot 0) = 13k.$$

An optimal edit sequence achieving this cost is simply to delete the six characters of TINTIN and then insert the seven characters of HADDOCK. For example (one valid minimal sequence):

1. Delete T (cost k),
2. Delete I (cost k),
3. Delete N (cost k),
4. Delete T (cost k),
5. Delete I (cost k),
6. Delete N (cost k).
(Now the string is empty; total cost so far $6k$.)
7. Insert H (cost k),
8. Insert A (cost k),
9. Insert D (cost k),
10. Insert D (cost k),
11. Insert O (cost k),
12. Insert C (cost k),
13. Insert K (cost k).

Total cost $= 6k + 7k = 13k$, which (by the LCS argument above) is optimal. □

Question 4. Let $W = \{w_1, w_2, \dots, w_n\}$ be a finite set of strings over a fixed alphabet Σ . An *edit center* for W is a string $C \in \Sigma^*$ such that the maximum edit distance from C to any string in W is as small as possible. The *edit radius* of W is the maximum edit distance from an edit center to a string in W .

A set of strings may have several edit centers, but its edit radius is unique.

$$\text{EditRadius}(W) := \min_{C \in \Sigma^*} \max_{w \in W} \text{Edit}(w, C)$$

$$\text{EditCenter}(W) := \arg \min_{C \in \Sigma^*} \max_{w \in W} \text{Edit}(w, C)$$

Describe and analyze an efficient algorithm to compute the edit radius of three given strings.

Solution. By the triangle inequality one has the lower bound

$$\text{LB} = \left\lceil \frac{\max_{i < j} \text{Edit}(w_i, w_j)}{2} \right\rceil$$

and the trivial upper bound

$$\text{UB} = \min_{t=1,2,3} \max_{s \neq t} \text{Edit}(w_s, w_t),$$

because any input string can be used as a center. We binary-search an integer radius r in $[\text{LB}, \text{UB}]$ and for fixed r decide whether there exists a string C with $\max_t \text{Edit}(w_t, C) \leq r$. The feasibility test for a fixed r is carried out by viewing any center C together with optimal edits from C to each w_t as a column-wise alignment. Each alignment column consumes either the next character of a word or a gap. In each column the best choice of the consensus character is one of the characters present in that column (choosing a character outside them can only increase per-string costs), so each column contributes 0 or 1 to the edit-distance of each string. Thus every column yields a cost vector in $\{0, 1\}^3 \setminus \{(0, 0, 0)\}$ and advances a subset of the three string indices by one. The feasibility question reduces to: is there a sequence of such columns that advances $(0, 0, 0)$ to (n_1, n_2, n_3) while the per-string costs never exceed r ?

Let us denote positions by (i, j, k) ($0 \leq i \leq n_1$, etc.) and remaining budgets by (a, b, c) with $0 \leq a, b, c \leq r$. Now to check feasibility, we use a BFS approach

Algorithm 1 Feasible(r)

```
Initialize queue and mark  $(0, 0, 0, r, r, r)$  as visited
while queue not empty do
  Pop state  $(i, j, k, a, b, c)$ 
  if  $i = n_1$  and  $j = n_2$  and  $k = n_3$  then
    return true
  end if
  for all nonempty  $P \subseteq \{1, 2, 3\}$  do
     $i' = i + \mathbf{1}_{1 \in P}$ 
     $j' = j + \mathbf{1}_{2 \in P}$ 
     $k' = k + \mathbf{1}_{3 \in P}$ 
    Let  $S$  be the multiset of consumed characters
    for all  $c^* \in S$  do
      Compute edits  $(\delta_1, \delta_2, \delta_3)$ 
      if  $a - \delta_1 \geq 0$  and  $b - \delta_2 \geq 0$  and  $c - \delta_3 \geq 0$  then
        if  $(i', j', k', a - \delta_1, b - \delta_2, c - \delta_3)$  not visited then
          Mark visited and push to queue
        end if
      end if
    end for
  end for
end while
return false
```

Overall algorithm: Compute pairwise distances $\text{Edit}(w_i, w_j)$ using standard two-string DP in time $O(n_i n_j)$ to get LB, UB. Binary-search integer r in $[\text{LB}, \text{UB}]$ and call Feasible(r). Return the smallest r for which Feasible(r) is true.

Complexity analysis: Let $n = \max\{n_1, n_2, n_3\}$. A single call to Feasible(r) explores states of the form (i, j, k, a, b, c) with $0 \leq i \leq n_1$, etc., and $0 \leq a, b, c \leq r$. Thus the number of possible states is at most

$$(n_1 + 1)(n_2 + 1)(n_3 + 1)(r + 1)^3 = O(n^3 r^3).$$

From each state we consider at most 7 nonempty subsets of strings whose positions are advanced and for each pattern at most 3 choices of consensus character, so the time per state is constant; hence one feasibility test runs in time $O(n^3 r^3)$. The binary search requires $O(\log(\text{UB} - \text{LB} + 1)) = O(\log n)$ calls, and the initial pairwise distances take $O(n^2)$ each (three such computations). Therefore total time is

$$O(n^3 r^3 \log n),$$

and space is $O(n^3 r^3)$ to store visited states. □

Question 5. Suppose it is nearing the end of the semester and you are taking n courses, each with a final project that still has to be done. Each project will be graded on the following scale: it will be assigned an integer number on a scale of 1 to $g \geq 1$, higher numbers being better grades. Your goal, of course, is to maximize your average grade on the n projects.

Now, you have a total of $H > n$ hours in which to work on the n projects cumulatively, and you want to decide how to divide up this time. For simplicity, assume H is a positive integer, and you will spend an integer number of hours on each project. So as to figure out how best to divide up your time, you have come up with a set of functions $\{f_i : i = 1, 2, \dots, n\}$ (rough estimates, of course) for each of your n courses; if you spend $h \leq H$ hours on the project for course i , you will get a grade of $f_i(h)$. (You may assume that the functions f_i are non-decreasing: if $h < h'$ then $f_i(h) \leq f_i(h')$.)

So the problem is: given these functions $\{f_i\}$, decide how many hours to spend on each project (in integer values only) so that your average grade, as computed according to the f_i , is as large as possible. In order to be efficient, the running time of your algorithm should be polynomial in n , g , and H ; none of these quantities should appear as an exponent in your running time.

Solution. We want to choose integers $h_1, \dots, h_n$ (hours on each course project) with

$$h_i \geq 1 \quad \text{and} \quad \sum_{i=1}^n h_i = H$$

to maximize the average grade $\frac{1}{n} \sum_{i=1}^n f_i(h_i)$. Since n is fixed, this is equivalent to maximizing the total grade $\sum_{i=1}^n f_i(h_i)$.

Step 1: Remove the lower bound $h_i \geq 1$. Give each project 1 hour upfront. Let

$$R := H - n \quad (\text{so } R \geq 1), \quad x_i := h_i - 1 \quad (\geq 0).$$

Then $\sum_i x_i = R$ and the objective becomes

$$\sum_{i=1}^n f_i(h_i) = \sum_{i=1}^n f_i(x_i + 1).$$

Define $g_i(x) := f_i(x + 1)$ for $x \in \{0, 1, \dots, R\}$.

Step 2: Dynamic programming (resource allocation DP). Define $dp[i][t]$ as the maximum total grade achievable using the first i courses with exactly t *extra* hours (i.e., over and above the mandatory 1 hour per course).

Base cases:

$$dp[0][0] = 0, \quad dp[0][t] = -\infty \text{ for all } t > 0.$$

(The value $-\infty$ denotes an impossible state.)

Transition: For $i = 1, 2, \dots, n$ and $t = 0, 1, \dots, R$,

$$dp[i][t] = \max_{x=0}^t \left(dp[i-1][t-x] + g_i(x) \right) = \max_{x=0}^t \left(dp[i-1][t-x] + f_i(x+1) \right).$$

Here, x is the number of extra hours assigned to course i (so $h_i = x + 1$).

Answer: The maximum total grade is $dp[n][R]$, hence the maximum average grade is

$$\frac{dp[n][R]}{n}.$$

If we also store which x achieves the maximum in each state, we can backtrack to recover the optimal allocation $(x_1, \dots, x_n)$ and thus $(h_1, \dots, h_n)$ with $h_i = x_i + 1$.

Running time and space: There are $(n+1)(R+1) = O(nH)$ states. Each transition tries $x = 0, \dots, t$, costing $O(H)$ per state in the worst case. Therefore the total running time is

$$O\left(\sum_{i=1}^n \sum_{t=0}^R (t+1)\right) = O(nR^2) = O(nH^2),$$

which is polynomial in n and H (and hence also polynomial in n, g, H as required). Space is $O(nR) = O(nH)$ if we store the full table $\square$

Question 6. Suppose you have a movable shop that you can take from one place to another. You usually take your shop to one of the two cities, say A and B , depending on whichever place has more demand. Suppose you have quite accurate projections for the earnings per day in both the cities for the next n days. However, you cannot simply go to the higher earning city each day because it takes one whole day and costs c to move from one city to the other.

For example, if you are in city A on day 5 and want to move to city B , then on day 6 you will have no earnings, you will pay a cost of c , and on day 7 you will have earnings of city B .

Design a polynomial-time algorithm that takes as input the moving cost c , the earnings per day in the two cities, say $a_1, a_2, \dots, a_n$ and $b_1, b_2, \dots, b_n$, and outputs a schedule for the n days that maximizes the total earnings. Assume that you can start in either of the two cities on day 1 without incurring any cost.

Solution. We have two cities A and B . If the shop is operated in city A on day i , it earns a_i ; if operated in city B on day i , it earns b_i . Moving from one city to the other takes one full day: on a moving day there are no earnings and we pay cost c .

DP state. Let

$DP_A[i] := \text{max earnings at the end of day } i \text{ if the shop is in city } A \text{ at the end of day } i$, and similarly define $DP_B[i]$.

Transitions. To end day i in city A , there are two possibilities:

- (a) We were already in A at the end of day $i - 1$ and we *do not move* on day i . Then we earn a_i on day i , giving value $DP_A[i - 1] + a_i$.
- (b) We were in B at the end of day $i - 1$ and we *move* from B to A on day i . Then we earn 0 on day i and pay cost c , giving value $DP_B[i - 1] - c$.

Hence, for $i \geq 2$,

$$DP_A[i] = \max\{DP_A[i - 1] + a_i, DP_B[i - 1] - c\}.$$

Similarly,

$$DP_B[i] = \max\{DP_B[i - 1] + b_i, DP_A[i - 1] - c\}.$$

Base cases. On day 1 we may start in either city at no cost, so

$$DP_A[1] = a_1, \quad DP_B[1] = b_1.$$

Answer and reconstruction. The optimal total earnings over n days is

$$\max\{DP_A[n], DP_B[n]\}.$$

To output an actual schedule, store for each i and each city which of the two options (attached above) attains the maximum, and backtrack from the better of $DP_A[n]$ and $DP_B[n]$ to recover for each day whether we stayed and earned in a city or moved.

Complexity. We compute two values per day, each in $O(1)$ time, for n days. Thus running time is $O(n)$ and space is $O(n)$ if we store backpointers (or $O(1)$ if only the optimal value is needed). $\square$

Question 7. Let $G = (V, E)$ be a directed graph with edge costs c_e for each $e \in E$. The edge costs may be negative. A designated vertex $t \in V$ is given as a sink. For every vertex $v \in V$, you are also given a finite value $d(v)$. It is claimed that for each vertex v , the value $d(v)$ equals the cost of a minimum-cost path from v to the sink t .

- (a) Design a linear-time algorithm (i.e., running in $O(m)$ time, where $m = |E|$) to verify whether this claim is correct.
- (b) Now assume that the given distances are indeed correct and that $d(v)$ is finite for all $v \in V$. Suppose you are given a different sink $t' \in V$, and you need to compute the minimum-cost distances $d'(v)$ from every vertex $v \in V$ to the new sink t' .

Give an algorithm that runs in $O(m \log n)$ time to compute these new distances, where $n = |V|$.

(Hint: Define a new edge cost function as follows. For each edge $e = (v, w)$, let $c'_e = c_e - d(v) + d(w)$. Is there a relationship between the costs of paths under the original cost function c and the modified cost function c' ?)

Solution. Let $G = (V, E)$ be a directed graph with edge costs c_e (possibly negative). A sink $t \in V$ is given, and for every vertex v we are given a finite value $d(v)$, claimed to be the minimum cost of a path from v to t .

(a) Verification in $O(m)$ time.

A necessary condition for correctness of shortest-path distances to sink t is:

$$d(t) = 0 \quad \text{and} \quad d(v) \leq c(v, w) + d(w) \quad \text{for every edge } (v, w) \in E.$$

(The inequality is the usual “triangle inequality” for shortest paths.)

It is also necessary that for every $v \neq t$, there exists an outgoing edge on some shortest path, i.e. an edge (v, w) such that

$$d(v) = c(v, w) + d(w).$$

Call such edges *tight*. If the claim is correct, then starting from any v and repeatedly following a tight edge must eventually reach t (this is just walking along a shortest path).

Algorithm (linear time).

- (1) Check $d(t) = 0$. If not, return **false**.
- (2) For each edge $(v, w) \in E$, check whether $d(v) \leq c(v, w) + d(w)$. If any edge violates this, return **false**.
- (3) Build the directed graph H of tight edges: include (v, w) in H iff $d(v) = c(v, w) + d(w)$.
- (4) Compute the set of vertices that can reach t using only tight edges: run a DFS/BFS from t in the *reverse* of H (i.e. traverse incoming tight edges). Let R be the set of visited vertices.
- (5) If $R \neq V$, return **false**; else return **true**.

Correctness. If the claim is correct, then for every edge (v, w) we must have $d(v) \leq c(v, w) + d(w)$, and also $d(t) = 0$. Moreover, for each $v \neq t$ there is a shortest path $v \rightsquigarrow t$ whose first edge (v, w) must be tight, and continuing similarly yields a tight-edge path from v to t ; hence every vertex can reach t in H , so $R = V$.

Conversely, if the algorithm returns **true**, then all edges satisfy $d(v) \leq c(v, w) + d(w)$, so for any path $P : v \rightsquigarrow t$ we get by repeated application that

$$d(v) \leq \text{cost}(P) + d(t) = \text{cost}(P),$$

i.e. $d(v)$ is a lower bound on the cost of *every* $v \rightsquigarrow t$ path. Also, since $v \in R$, there exists a tight-edge path $P^* : v \rightsquigarrow t$ in H , and along each tight edge we have equality, so telescoping gives $\text{cost}(P^*) = d(v)$. Hence $d(v)$ equals the true minimum path cost from v to t .

Running time. Steps (1)–(3) scan vertices/edges once: $O(n + m)$. Step (4) is one BFS/DFS on H (at most m edges), so $O(n + m)$. Thus total time is $O(m)$.

(b) Recompute distances to a new sink t' in $O(m \log n)$.

Intuition. The difficulty is that edge costs c_e may be negative, so we cannot run Dijkstra directly. But the given distances $d(\cdot)$ to the old sink t act like a *potential function*: since $d(v) \leq c(v, w) + d(w)$ for every edge, the quantity $c(v, w) - d(v) + d(w)$ is always nonnegative. Reweighting edges by this potential removes negative costs, while preserving the relative costs of all paths up to an endpoint-dependent constant. This is exactly the trick used in Johnson's algorithm.

Reweighting. Define a new edge cost function: for each edge $e = (v, w)$,

$$c'_e := c_e - d(v) + d(w).$$

From $d(v) \leq c(v, w) + d(w)$ we get $c'_e \geq 0$ for all edges, so all reweighted costs are nonnegative.

Key path identity. For any directed path $P : v = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k = t'$,

$$\sum_{i=0}^{k-1} c'(v_i, v_{i+1}) = \sum_{i=0}^{k-1} (c(v_i, v_{i+1}) - d(v_i) + d(v_{i+1})) = \text{cost}(P) - d(v) + d(t'),$$

because the $d(\cdot)$ terms telescope.

Thus for every v ,

$$\min_{P: v \rightsquigarrow t'} \text{cost}(P) = \min_{P: v \rightsquigarrow t'} \sum_{e \in P} c'(e) + d(v) - d(t'). \quad (1)$$

Algorithm.

- (1) Compute all c'_e in one pass over edges.
- (2) Consider the reverse graph G^{rev} (reverse every edge). For each reversed edge (w, v) , assign it cost $c'(v, w)$.
- (3) Run Dijkstra's algorithm from source t' in G^{rev} (all edge weights are ≥ 0). Let $\Delta(v)$ be the distance found from t' to v in G^{rev} . (Equivalently, $\Delta(v)$ is the minimum c' -cost of a path from v to t' in G .)
- (4) Output

$$d'(v) = \Delta(v) + d(v) - d(t') \quad \text{for all } v \in V.$$

Complexity. Reweighting costs $O(m)$. Dijkstra on n vertices and m edges runs in $O(m \log n)$ time (using a binary heap). Total time is $O(m \log n)$. $\square$

Question 8. The Hadamard matrices $H_0, H_1, H_2, \dots$ are defined as follows:

- H_0 is the 1×1 matrix $H_0 = [1]$.
- For $k > 0$, H_k is the $2^k \times 2^k$ matrix

$$H_k = \begin{bmatrix} H_{k-1} & H_{k-1} \\ H_{k-1} & -H_{k-1} \end{bmatrix}.$$

Show that if v is a column vector of length $n = 2^k$, then the matrix-vector product $H_k v$ can be calculated using $O(n \log n)$ operations. Assume that all the numbers involved are small enough that basic arithmetic operations like addition and multiplication take unit time.

Solution. Let $n = 2^k$ and let $v \in \mathbb{R}^n$ be a column vector. Write v as two halves:

$$v = \begin{bmatrix} v^{(0)} \\ v^{(1)} \end{bmatrix}, \quad \text{where } v^{(0)}, v^{(1)} \in \mathbb{R}^{n/2}.$$

Using the block definition

$$H_k = \begin{bmatrix} H_{k-1} & H_{k-1} \\ H_{k-1} & -H_{k-1} \end{bmatrix},$$

we get

$$H_k v = \begin{bmatrix} H_{k-1} & H_{k-1} \\ H_{k-1} & -H_{k-1} \end{bmatrix} \begin{bmatrix} v^{(0)} \\ v^{(1)} \end{bmatrix} = \begin{bmatrix} H_{k-1}(v^{(0)} + v^{(1)}) \\ H_{k-1}(v^{(0)} - v^{(1)}) \end{bmatrix}.$$

Therefore the product $H_k v$ can be computed recursively by:

- (1) Compute $s := v^{(0)} + v^{(1)}$ and $t := v^{(0)} - v^{(1)}$ (componentwise).
- (2) Recursively compute $y^{(0)} := H_{k-1}s$ and $y^{(1)} := H_{k-1}t$.
- (3) Output $y = \begin{bmatrix} y^{(0)} \\ y^{(1)} \end{bmatrix}$.

Algorithm (Hadamard transform).

Algorithm 2 HADAMARDMULT(v)

Input: $v \in \mathbb{R}^n$ where n is a power of 2

Output: $H_k v$

if $n = 1$ **then**

return v $\{H_0 = [1]\}$

end if

Split $v = \begin{bmatrix} v^{(0)} \\ v^{(1)} \end{bmatrix}$ with $|v^{(0)}| = |v^{(1)}| = n/2$

$s \leftarrow v^{(0)} + v^{(1)}$ $\{n/2 \text{ additions}\}$

$t \leftarrow v^{(0)} - v^{(1)}$ $\{n/2 \text{ additions}\}$

$y^{(0)} \leftarrow \text{HADAMARDMULT}(s)$

$y^{(1)} \leftarrow \text{HADAMARDMULT}(t)$

return $\begin{bmatrix} y^{(0)} \\ y^{(1)} \end{bmatrix}$

Correctness. We prove by induction on k (equivalently on $n = 2^k$) that HADAMARDMULT(v) returns $H_k v$. For $k = 0$ we have $n = 1$ and the algorithm returns v , which

equals H_0v since $H_0 = [1]$. Assume the claim holds for $k - 1$. For k , the algorithm forms $s = v^{(0)} + v^{(1)}$ and $t = v^{(0)} - v^{(1)}$, and by the induction hypothesis it returns $y^{(0)} = H_{k-1}s$ and $y^{(1)} = H_{k-1}t$. Hence the output is

$$\begin{bmatrix} H_{k-1}(v^{(0)} + v^{(1)}) \\ H_{k-1}(v^{(0)} - v^{(1)}) \end{bmatrix} = H_k v,$$

using the block-matrix multiplication shown at the start. This completes the induction.

Running time. Let $T(n)$ be the number of arithmetic operations to compute H_kv for $|v| = n$. Forming s and t costs n additions/subtractions total, and we make two recursive calls on size $n/2$:

$$T(n) = 2T(n/2) + O(n), \quad T(1) = O(1).$$

By the Master Theorem, $T(n) = O(n \log n)$. Therefore H_kv can be computed using $O(n \log n)$ operations. $\square$